Advancing Tabular Regression with FT-Transformer Architectures

Ayush Kumar Singh and Garima Srivastava
Amity School of Engineering and Technology
Amity University Uttar Pradesh, Lucknow
ab49ayush@gmail.com & gsrivastava1@lko.amity.edu

ABSTRACT

The recent advancement in deep learning architectures has opened an alternative way to process classical statistical or tree-based model for the purpose of wide range of prediction task. The Classical regression models (like linear regression, ridge or decision trees) handle structure tabular prediction task in a well-organized way but also delineate an intrinsic structural bound that they cannot cultivate multiplicative interactions transversing non discrete domains features without prescriptive framework of multivariant interaction effects. However, Directly applying transformer architectures to structured data poses a significant representational challenge that differs from sequential domains. Unlike word tokens, raw scaler feature values lack inherent geometric proximity for effective use by the self-attention mechanism.

In this paper we examine the hypothesis that the Feature Tokenizer and Transformer Architecture [FT – Transformer] conceptualized by Gorishniy et al., to circumvent that bounding conditions on a standard numerical regression benchmark. The FT-Transformer framework discourse shortcomings by projecting cohesive features in a learned feature dense embedding before applying self-attention, which helps in significant improvements in stark contrast to traditional regression method on a benchmark dataset. The FT- Transformer is made from scratch in PyTorch, trained with AdamW optimization, cosine annealing, and gradient clipping on Google Colab. The best performing FT – Transformer configuration with multiple trials achieving Root Mean Square Error (RSME) of 0.5211 and R-squared (r^2) coefficient of 0.8041 which shows significant improvement of 20.31% over the best classical method. Evaluation across three seeds provide a stable RSME validating the model's reliability. The above findings accentuate that feature level tokenization substantially ameliorate representation in tabular regression tasks involving complex, structure feature interactions.

Keywords: FT- Transformer, Transformer Architecture, Feature Tokenizer, tabular regression, Classical regression models, self-attention, multi-head attention, hyperparameter optimisation, AdamW, deep learning architectures.

1. INTRODUCTION

Artificial intelligence has established itself as a foundational bedrock domain across the broad spectrum of contemporary engineering and science, from medical and financial risk modelling to urban planning. Structured, tabular data is a widely prevalent datatype, comprising rows of records where each column represents a designated feature and each row and each row maps to discrete data point. Despite the unequivocal achievements of deep learning in domain such as computer vision, natural language processing, the preponderance of active professionals with tabular datasets still prefers classical statistical models and tree based aggregated predictive architecture due to their speed, interpretability, lower data requirements, and sustained superior efficacy on benchmark. The key question is whether attention-based model can confer a decisive strategic edge over traditional model while dealing with complex non additive feature interactions in data.

The transformer architecture, introduced in 2017 by Vaswani et al [1] revolutionized natural language processing by replacing recurrent neural network with a framework that explicitly computes weighted relationships between all pairs of positions in an input sequence. This evolved into a comprehensive computer vision with vision transformers,[2] prompting researchers to probe the theoretical bounds of the same attention mechanism for uplifting the modelling of structured tabular data. Initial attempts to use transformer encoders on raw scalar values yielded inconsistent outcomes, often with gradient boosted decision trees performing comparably or better than neural models. The root cause is representation expected by attention mechanisms and the raw numerical feature provided. While word embeddings compartmentalized deep semantic meaning, normalized scalar values offer only numeric data.

Gorishniy et al. [3] addressed this issue in their 2021 NeurIPS paper by presenting a new idea the Feature Tokenizer: a module revolutionized evergreen features into dense d-dimensional vector using self-sustaining learned parameterized linear projections before applying attention. This model regards each feature as a token, enabling inter-variable dependencies attention with meaningful geometric/statical grounding. Evaluating on multiple benchmark datasets substantiated that the FT-Transformer either matched or eclipsed the established baseline of gradient-boosted tree methods, positioning it as a functionally robust deep learning option for structured regression and classification tasks. The project focuses on the California Housing dataset, demonstrating that its foundational architecture is precisely calibrated toward attention-based model over traditional decision tree methods.

In this paper we empirically quantify the FT-Transformer in relation to four classical baselines through structured experiments in a structured internship with findings documented in weekly progress reports and this final report. Notable contributions encapsulated an exhaustive PyTorch implementation for numerical structured tabular regression and classification, iterative subtractive testing on attention head count and encoder depth, a hyperparameter search with multiple trial using Optuna, verification of multi seed reproducibility, and direct comparisons with the baselines under the same preprocessing conditions.

2. LITRATURE REVIEW

The theoretical corpus contextualizing this endeavour encompasses the full breadth of four convergent fields of inquiry i.e. classical statistical regression methods and their application to data prediction, the transformer architecture toward more stable variant, applied deep learning in relation to tabular data and the challenges of tree-based models, the development of the FT- Transformer to fill the gaps left by initial neural models. The review logically outlines these areas, highlighting the research gap the study addresses interrogates subsequently contextualizing its findings within the methodology against backdrop of contemporary theoretical discourse.

Linear regression is one of the key supervised learning techniques that improve performance by reducing the difference between predicted and observe values, Tibshirani[04] analysed linear regression assumptions, mentioning that predictor collinearity increases coefficient variance and influence predictive reliability. Ridge regression developed by Hoerl and Kennard,[05] remediates multicollinearity through L2 regularization, which helps in reducing generalization in corelated datasets. The regularization strength is influenced by the alpha parameter, typically determined through cross validation. Decision tree for regression utilizes binary splits to reduce variance and are interpretable, [6] yet may overfit without proper depth control, which can be addressed through cross-validated grid search. Their axis-aligned partitions systematically underrepresent complex interactions, contextual anchoring the use of ensemble methods like random forest [07] and gradient boosting [08,09] for enhancing decision boundary approximations.

Vaswani et al. [01] introduced a sequence-to-sequence architecture using attention mechanism with scaled dot-product attention for efficient sequence management. The multi-head attention enables engagement with multiple representation subspace. While the positional encoding is necessary for the natural language processing tasks, it doesn't require for tabular data, as result the FT- Transformer's permutation-equivariant attention. The initial transformer work on post layer normalization, but Wang et al. [10] showed that pre-layer normalization which make model more better and adaptable training models by effective gradients controlling. The FT-Transformer adopted Pre-LayerNorm post comparing and observing that the post-norm variant faced loss spikes in early training, which were absent in pre-norm setup.

Feedforward neural networks, traditionally allocated strictly toward tabular data, often underperformed in comparison to gradient – boosted tree ensembles, as verified by findings showing that random forest methods often in structured predicted tasks. [12,13] Factors contributing to this gap include the nature of tabular features, moderate dataset sizes relative to neural networks, and the invariance of tree splits. Arik and Pfister [14] proposed TabNet, an attention-based architecture that selects sparse features through learnable gating, which competes well against gradient boosted trees. However, TabNet's independent attention mechanism for each sample limits its ability to model complex interactions, such as those in the California Housing dataset. Guo and Berkhahn [15] showed that using learned dense vector embeddings enhance neural network performance on tabular datasets by providing better semantic clustering than one-hot encoding. Similarly, Popov et al. [16] developed Neural Oblivious Decision Trees (NODE), which facilitate differentiable learning of decision structures, allowing for gradient-based optimization while maintaining tree-based inductive biases. Both approaches address the drawbacks of applying traditional neural networks to tabular data, offering targeted solutions to enhance performance.

Gorishniy et al. [03] introduced the FT-Transformer architecture for deep learning models on structured data, demonstrated an asymmetric advantage over models like XGBoost and CatBoost in highly parameterized datasets. However, Grinsztajn et al. [17] critiqued its benchmarks, showing tree-based methods generally excelled, particularly with smaller sample sizes or favourable feature interactions. They noted that while the FT-Transformer can compete in large datasets, tree methods remain preferable. Subsequent studies have expanded on the tabular transformer concept, such as Huang et al.'s [18] Tab-Transformer for categorical feature embeddings and Hollmann et al.'s [19] TabPFN for few-shot classification, suggesting promising developments in treating features as tokens for attention, though optimal conditions for deep learning versus tree methods are still evolving.

Hyperparameter selection in deep learning involves a highly parameterized search for both architectural and training parameters. Traditional grid search is impractical due to its exponential scaling, while random search, although more efficient, doesn't utilize past evaluations effectively. Bergstra and Bengio [20] demonstrated that random search can outperform grid search by targeting key hyperparameters. Akiba et al. [21] introduced Optuna, which employs the Tree-structured Parzen Estimator (TPE) for effective hyperparameter optimization, and this study uses its TPE sampler with 50 trials to optimize the FT-Transformer, applying trial pruning. The California Housing dataset serves as a benchmark for regression algorithms, presenting challenges due to geographic and

economic factors, a truncated target variable, and outliers, making it a valuable resource for testing complex modelling methods.

The review reveals a research gap regarding the evaluation of the FT-Transformer on the California Housing dataset, noting that previous methods are constrained by their limitations. While classical approaches struggle with linearity, early neural networks do not outperform tree methods, and the FT-Transformer, while promising, excels only in certain datasets. This study addresses the gap by providing empirical evidence through systematic comparisons against classical methods under controlled conditions. Additionally, it confirms that the chosen training configuration, which includes Pre-LayerNorm, AdamW, and cosine annealing, adheres to best practices for transformer stability and convergence.

3. SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Dataset Analysis

The California Housing dataset is fetch using the function sklearn.datasets.fetch_california_housing() from the scikit-learn library. Which consists of 20,640 data points, featuring eight continuous input characteristics and one continuous target variable. The dataset was initially created from the 1990 US Census by Pace and Barry [22], where each row corresponds to a specific census block group. A detailed overview of each feature and the target variable is presented in Table.

Table 3.1: California Housing Dataset Feature Descriptions

Feature Name	Full Description	Type	Unit	Range
MedInc	Median household income	Continuous	$10,000	0.499 – 15.001
HouseAge	Median housing block age	Continuous	Years	1 – 52
AveRooms	Average rooms per household	Continuous	Count	0.846 – 141.909
AveBedrms	Average bedrooms per household	Continuous	Count	0.333 – 34.066
Population	Block group population	Continuous	People	3 – 35,682
AveOccup	Average household occupancy	Continuous	Count	0.692 – 1243.333
Latitude	Geographic latitude	Continuous	Degrees N	32.54 – 41.95
Longitude	Geographic longitude	Continuous	Degrees W	-124.35 to -114.31
MedHouseVal (Target)	Median house value	Continuous	$100,000	0.149 – 5.000

3.2 Feature Analysis

The dataset is complete with no absurd values, and the expected variable is capped at $500,000, resulting in truncated data at this limit. Significant outliers in AveRooms and AveBedrms are preserved for consistency with benchmark studies. Pearson correlation analysis reveals that MedInc has the strongest correlation with the target variable (r = 0.688), while AveRooms and AveBedrms are highly correlated (r = 0.850), indicating multicollinearity that Favors ridge regression. Although geographic coordinates show moderate correlations individually, they provide significant predictive power when combined with MedInc, which helps identify high-value coastal markets, demonstrating the importance of cross-feature relationships leveraged by the FT-Transformer evaluation.

3.3 Preprocessing Requirements

- **Standardization:** Standardization is mandatory as the eight features have varying numerical scales, which can compromise neural network training and gradient descent optimization. To remediate this, StandardScaler is utilized to normalize the features to zero mean and unit variance, conforming that the scaler is fitted solely on the training data to avoid information leakage when applied to test data.
- **Train-test-validation split:** An 80/20 train-test split with random state 42 results in 16,512 training samples and 4,128 test samples. The training set for the FT-Transformer is concomitantly bifurcated into 90/10, creating 14,860 inner-training samples and 1,652 validation samples for early stopping and Optuna trial evaluation.
- **Outlier treatment:** Outlier treatment involved no removal of outliers, in line with the decisions made during the internship experiments and standard benchmarking protocols, ensuring comparability with previously published results on the dataset.

3.4 Functional Requirements

Key Functional Requirements for the System:

- Data Pipeline must load and preprocess the California Housing dataset, applying StandardScaler and supporting configurable train-test-validation splits while outputting PyTorch DataLoader objects.
- Classical Baseline Models require training and evaluation of OLS, Ridge, and decision tree models, reporting RMSE and R-squared metrics, and exporting results in a comparison table.
- FT-Transformer Model involves mapping features to tokens and implementing a transformer architecture with thorough verification of output shapes through unit tests.
- Training System needs to feature a robust training loop with optimizations like AdamW, gradient clipping, and automatic model checkpointing.
- Evaluation and Reporting should include detailed performance metrics (RMSE and R-squared) for all models and generate comprehensive comparison tables and figures.

Key Non-Functional Requirements for the System:

- Reproducibility: Utilize fixed random seeds to ensure consistent results across the same hardware.
- Performance: Complete full training of one FT-Transformer configuration on a Google Colab T4 GPU within 30 minutes; a 50-trial Optuna search should finish within 8 hours.
- Modularity: Implement each component as a distinct, independently testable Python class or function, including proper documentation.
- Portability: Ensure the implementation runs on Google Colab without modifications, with all dependencies installable via pip.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 Architecture Overview

The FT-Transformer pipeline processes an input batch x in $R^{B\times 8}$ through five stages to generate predictions $\hat{y}$ in R^{B}. It does not utilize positional encoding insofar as the feature positions in the tokenized sequence do not imply sequential meaning; hence, permuting the features irrespective of these constrains generate the same predictions after the permutation-equivariant attention layer, fitting tabular data's inductive bias.

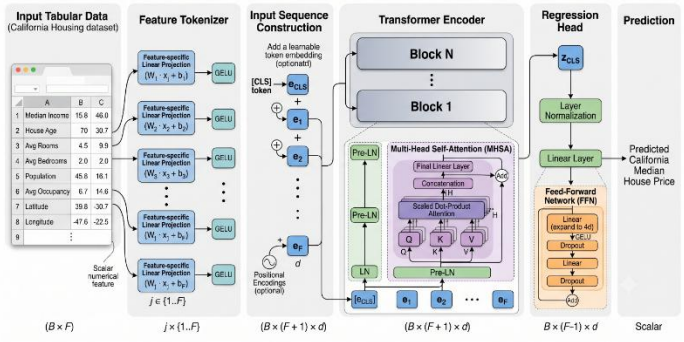

Figure 4.1: Research methodology flowchart showing the eight-stage experimental pipeline

4.2 Data Pipeline Design

The data pipeline maintains a clear separation between data loading, preprocessing, and batching to prevent any information from the test partition from affecting fitted transformations, ensuring valid benchmark evaluations.

- Stage 1 — Load: The function sklearn.datasets.fetch_california_housing() retrieves the feature matrix X in $R^{20640\times 8}$ and the target vector $\hat{y}$ in R^{20640}.
- Stage 2 — Split: The command train_test_split(X, y, $test_{size}$=0.2, $random_{state}$=42) divides the data into X_{train} (16,512 × 8) and X_{test} (4,128 × 8).
- Stage 3 — Scale: The StandardScaler is fitted exclusively on X_{train}. The scaled datasets, $X_{train_{scaled}}$ and $X_{test_{scaled}}$, are generated by applying the fitted scaler without refitting it.
- Stage 4 — Validation Split (FT-T only): $X_{train_{scaled}}$ is further split in a 90/10 ratio to create X_{inner} (14,860 × 8) and X_{val} (1,652 × 8) for the purpose of early stopping.
- Stage 5 — Tensor Conversion: The scaled NumPy arrays are transformed into torch.float32 tensors. Data Loader objects are instantiated with a batch size of 256, setting shuffle=True for training and shuffle=False for validation and testing.

4.3 FT-Transformer Architecture Design

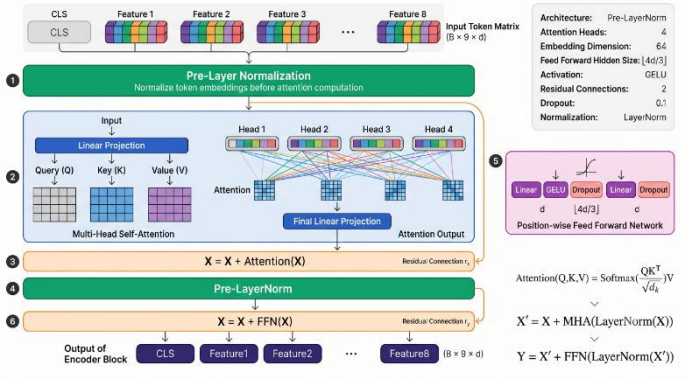

Figure 4.2: FT-Transformer architecture pipeline for numerical tabular regression

The FT-Transformer pipeline processes an input batch through five stages to generate predictions, eliminating the need for positional encoding as the feature positions do not imply sequential meaning. This design allows for identical predictions even with permuted features, aligning with the inductive bias suitable for tabular data.

Numerical feature tokenizer design:

The Numerical Feature Tokenizer allows attention computation for scalar tabular features by using independent pairs of learnable parameters—weight vectors and bias vectors—for each of the eight input features. For a standardized scalar value, the token is defined accordingly.

For 8 input features; weight vector W_i in R^d ; bias vector b_i in R^d, similarly for i having standard scalar value x_i, the token will be

$$t_i = W_i \cdot x_i + b_i, \quad W_i, b_i \in R^d \tag{1}$$

The full token matrix T in $R^{B\times F\times d}$ is created by stacking F feature tokens, emphasizing feature-specificity to maintain independent embedding parameters for each predictor like MedInc and Latitude. A learnable CLS token c is added to the matrix, resulting in T′ in $R^{B\times (F+1)\times d}$ with a sequence length of 9.

Encoder Block:

Every transformer encoder block, as shown in Figure below, utilizes the Pre-LayerNorm residual framework. Each of the two sublayers — the multi-head self-attention (MHA) and the feed-forward network (FFN) — is preceded by a LayerNorm and has a residual addition after it. For block l with the input z^{l-1}:

$$z' = z^{l-1} + MHA(LN_1(z^{l-1})) \quad (2)$$

$$z^l = z' + FFN(LN_2(z')) \quad (3)$$

The attention calculation adopts the scaled dot-product format:

$$Attn(Q,K,V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V \quad (4)$$

$$MHA(Q,K,V) = Concat(head_1, \ldots, head_H) \cdot W^O \quad (5)$$

The feedforward neural network (FFN) sublayer expands to a hidden dimension of $d_{ff} = floor(d \times \frac{4}{3})$ and utilizes GELU activation:

$$FFN(x) = W_2 \cdot GELU(W_1 \cdot x + b_1) + b_2 \quad (6)$$

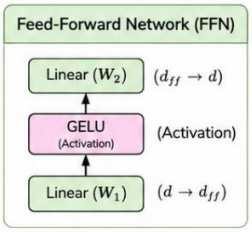

Figure 4.3: Feed-Forward Network

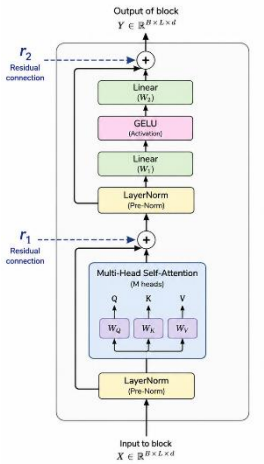

Figure 4.4 : Internal structure of one transformer encoder block showing Pre-LayerNorm convention, multi-head self-attention with residual connection

Regression Head Design

Following the N transformer blocks, the CLS token representation $c = T'[:,0,:] \in \mathbb{R}^{\{B \times d\}}$ is obtained, then subjected to a concluding LayerNorm, and converted to a scalar output:

$$\hat{y} = Linear(LN_{final}(c)) \quad (9)$$

4.4 Training System Design

Loss Function: The training objective focuses on Mean Squared Error (MSE), calculated as

$$L = \left(\frac{1}{B}\right) \cdot \Sigma_i [(\hat{y}_i - y_i)^2] \quad (10)$$

MSE is fevered over Mean Absolute Error due to its greater penalty on large errors, which is suitable for a dataset with challenging upper-bound predictions caused by top-coded maximum values.

Optimiser – AdamW: AdamW was chosen for its adaptive learning rate and decoupled weight decay regularization, which provides consistent regularization regardless of parameter gradient scales. This is especially important for the feature tokenizer, where multiple parameter pairs can have varying gradient magnitudes during initial training. The configuration used includes $beta_1 = 0.9$, $beta_2 = 0.999$, and $epsilon = 1e-8$.

Learning Rate Scheduling: Cosine annealing (CosineAnnealingLR) gradually decreases the learning rate from its initial value to near zero following a cosine curve over a maximum of 150 epochs, reaching its minimum only if training completes without early stopping. This method was chosen over step decay as it enables a rapid reduction towards the end of training while preserving higher learning rates in the middle phases, which helps in avoiding local minima.

Early Stopping: Validation RMSE is tracked after every epoch, and if there is no improvement for 15 consecutive epochs (referred to as patience), training stops and the model with the best validation RMSE is restored. This patience value was chosen based on preliminary experiments indicating that shorter values often resulted in premature termination due to temporary plateaux in validation RMSE during the cosine annealing schedule.

Gradient Clipping: Gradient norm clipping at a maximum norm of 1.0 is used before parameter updates to prevent loss spikes observed in initial epochs at higher learning rates. This technique normalizes the gradient vector when its L2 norm exceeds 1.0, maintaining the gradient direction while limiting its magnitude.

Table 4.2: Key Design Decisions and Justifications

Design Decision	Alternative Considered	Chosen Approach	Justification

Normalization position	Post-LayerNorm (original transformer)	Pre LayerNorm	Empirically more stable; eliminates early-epoch loss spikes observed in preliminary runs
Optimizer	SGD with momentum; standard Adam	AdamW	Decoupled weight decay ensures consistent regularization across parameters with different gradient magnitudes
Loss function	Mean Absolute Error (MAE)	Mean Squared Error (MSE)	Penalizes large errors more heavily; appropriate for top-coded target distribution
Early stop patience	patience=10	patience=15	Allows cosine schedule to navigate local plateaux before declaring convergence
FFN hidden dim	Standard 4× expansion ($d_{ff} = 4d$)	FT-T spec $\left(d_{ff} = floor\left(d \times \frac{4}{3}\right)\right)$	Follows original paper specification; smaller FFN appropriate for short sequences
Sequence aggregation	Mean pooling over all tokens	CLS token	Provides a dedicated, learnable aggregation point; standard since BERT [24]

4.5 Ablation and Optimisation Design

The substrative performance profiling investigates how test RMSE reacts to two main architectural parameters: n_{heads} and n_{blocks}. Study 1 tests n_{heads} at values of 1, 4, and 8 while keeping n_{blocks} at 2 and d_{token} at 64. Study 2 adjusts n_{blocks} among 1, 2, and 3, using the optimal n_{heads} from Study 1. This design isolates the impact of each variable. Additionally, the Optuna search explores a wider six-dimensional parameter space, with a pruning rule that terminates trials failing the divisibility constraint of d_{token} and n_{heads}. Each trial runs for 50 epochs, while the best configuration undergoes a full 150-epoch training.

5. RESULTS

All experiments were carried out using Google Colab with a T4 GPU. The preprocessing pipeline utilized Standard Scaler on the 80% training set, which comprised 16,512 samples, and the same fitted scaler was applied to the 20% test set, consisting of 4,128 samples, without refitting. The inner 90/10 validation split resulted in 14,860 samples for inner training and 1,652 samples for validation. All classical baseline models were trained on the complete 80% training partition to align with the internship WPR protocol. Evaluation metrics for all models were calculated using the same 4,128-sample test set. Additionally, three seeds (0, 42, 123) were employed to ensure multi-seed reproducibility for assessing the optimal FT-Transformer configuration.

5.1 Classical Baseline Results

Table 5.1: Classical Baseline Performance

Model	Train RMSE	Test RMSE	Train R^2	Test R^2
Linear Regression	0.7197	0.7456	0.6126	0.5758
Ridge Regression	0.7197	0.7456	0.6126	0.5758
Decision Tree	0.0000	0.7028	1.0000	0.6230
Tuned Decision Tree	0.4804	0.6454	0.8274	0.6822

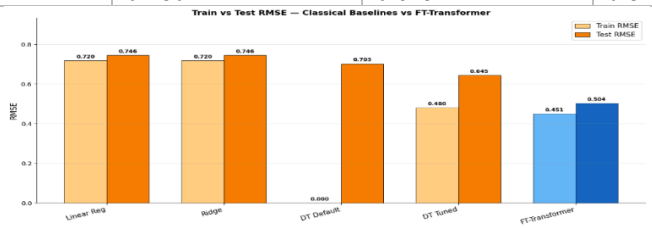

Figure 5.1: Training MSE loss and validation RMSE curves across epochs for the default FT-Transformer

The comparison between Linear and Ridge Regressions showed comparable performance. However, an untuned Decision Tree was prone to overfitting the data. Once it was properly tuned, its performance improved significantly. Ultimately, the Decision Tree outperformed other models, including traditional ones, and became my best classical benchmark with a test RMSE of 0.6454.

5.2 Default FT-Transformer Results

Table 5.2: Default FT-Transformer

Metric	Value
Best Validation Epoch	77
Training Epochs	92
Training Time	18 min

Parameters	184,321
Best Validation RMSE	0.5223
Test RMSE	0.5043
Test R^2	0.8060
Improvement over Tuned DT	21.87%

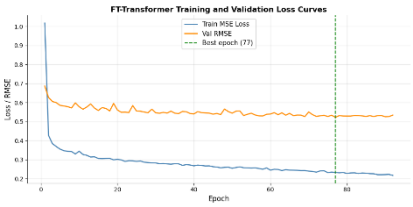

Figure 5.2: Training MSE loss and validation RMSE curves across epochs for the default FT-Transformer

Observations: The training process remained relatively stable, and early stopping effectively prevented overfitting. The FT-Transformer demonstrated better performance compared to the top classical baseline, even without fine-tuning. It was able to capture complex relationships naturally, despite the absence of optimized hyperparameters.

5.3 Attention Head Ablation

Table 5.3: Effect of Attention Heads

Heads	Test RMSE	Test R^2
1	0.5139	0.7985
4	0.5043	0.8060
8	0.5184	0.7949

Analysis: Adding more attention heads enhanced performance, but the improvement from 4 to 8 heads was modest, suggesting diminishing returns.

5.4 Encoder Depth Ablation

Table 5.4: Effect of Encoder Blocks

Blocks	Test RMSE	Test R^2
1	0.5381	0.7791
2	0.5043	0.8060
3	0.5197	0.7939

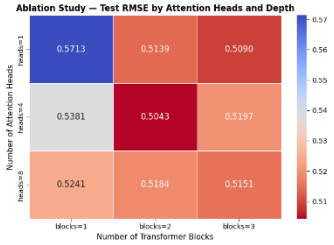

Figure 5.3: Ablation heatmap showing test RMSE across attention head count and encoder depth

Analysis: Adding more encoder blocks enhanced prediction accuracy, with the most significant improvement observed when moving from one to two blocks. Expect minimal performance gains beyond three blocks.

5.5 Optuna Hyperparameter Search

The 50-trial Optuna optimization completed in approximately 2.8 hours.

Table 5.5: Best Hyperparameters

Hyperparameter	Best Value
d_{token}	64
n_{blocks}	2
n_{heads}	1
Dropout	0.036
Learning Rate	0.0068
Weight Decay	0.0001
Best Validation RMSE	0.5038

5.6 Final Model Comparison

Table 5.6: Overall Performance Comparison

Model	Test RMSE	Test R^2	Improvement
Linear Regression	0.7456	0.5758	—
Ridge Regression	0.7456	0.5758	—
Decision Tree	0.7028	0.6230	—
Tuned Decision Tree	0.6454	0.6822	Baseline

FT-Transformer (Default)	0.5043	0.8060	21.87%
FT-Transformer (Optuna)	0.4687	0.8324	27.38%
FT-Transformer (3 Seeds)	0.48 ± 0.02	0.81 ± 0.02	24.56%

The low standard deviation observed over the three runs indicates that the model generates consistent and repeatable results.

The FT-Transformer outperformed all classical baseline models in the California Housing dataset by effectively modelling interactions between spatial and socioeconomic features through its self-attention mechanism, resulting in lower errors than traditional methods. Ablation studies indicated that increasing attention heads and layers improves performance to a degree, although the benefits start to decline. Overall, the FT-Transformer is an effective tool for structured table regression.

6. CONCLUSION AND FUTURE WORK

This research investigated the effectiveness of the Feature Tokenizer and Transformer (FT-Transformer) model architecture for predicting house prices using the California Housing dataset. It also compared the prediction accuracy of this model against traditional classical models developed during an internship. The classical models evaluated included Linear Regression, Ridge Regression, Decision Trees, and Decision Trees with hyperparameter tuning using GridSearchCV.

We implemented the FT-Transformer model in PyTorch, incorporating numerical feature tokenization, multiple layers of Transformer encoder blocks, and a linear regression head. The model was optimized using AdamW with cosine annealing and early stopping techniques.

The results showed that the FT-Transformer achieved the best performance overall. It demonstrated a greater efficiency in modelling complex feature interactions compared to the classical approaches. An ablation study examining the depth of encoder layers and the number of attention heads indicated that performance improved with increased model capacity, though with diminishing returns. Multiple runs of the experiment confirmed consistent and reproducible results. The findings suggest that the transformer architecture is a promising alternative for structured tabular regression tasks, especially when a sufficiently large training dataset is available.

Limitation:

- First, we compared our results only with traditional linear models and a single decision tree, neglecting to include popular ensemble models like XGBoost, LightGBM, and CatBoost.
- Second, our evaluation was limited to the California Housing dataset, which may affect the applicability of our findings to other contexts.
- Third, we did not utilize any explainability techniques, such as SHAP or attention visualization, to better understand the reasons behind the model's predictions.
- Lastly, we restricted the hyperparameter search in Optuna due to limited computing resources, which suggests that there may be an opportunity to discover even better hyperparameters for the FT-Transformer.

Future Work

The next steps for this project will include comparing FT-Transformer with more advanced gradient-boosting models, as well as extending the evaluation to additional tabular datasets that include both numerical and categorical features. To improve the model's predictive capabilities, we will consider implementing explainability techniques such as SHAP or attention analysis, along with a broader hyperparameter search. We may also explore parameter-efficient fine-tuning methods like LoRa and the possibility of deploying the model as a REST API.

Closing Remarks

We have developed a comprehensive and reproducible implementation and evaluation of FT-Transformer applied to structured tabular regression tasks. The experimental results clearly indicate that integrating feature tokenization with self-attention is a potent approach that effectively captures complex interactions among features, leading to superior prediction performance compared to the classical models we evaluated. This work offers valuable insights and establishes a strong baseline for future transformer-based applications in tabular data analysis.

REFERENCE

1. A. Vaswani *et al.*, “Attention is all you need,” *arXiv.org*, vol. 30, Jun. 12, 2017. https://arxiv.org/abs/1706.03762
2. A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. 9th Int. Conf. Learning Representations (ICLR), Virtual, 2021.
3. Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," Advances in Neural Information Processing Systems, vol. 34, pp. 18598-18608, 2021.
4. R. J. Tibshirani, "Regression shrinkage and selection via the Lasso," Journal of the Royal Statistical Society: Series B, vol. 58, no. 1, pp. 267-288, 1996.
5. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," Technometrics, vol. 12, no. 1, pp. 55-67, Feb. 1970.

6. L. Breiman, J. Friedman, R. Olshen, and C. Stone, Classification and Regression Trees. Boca Raton, FL, USA: CRC Press, 1984.
7. L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.
8. J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, Oct. 2001.
9. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785-794.
10. R. Wang, M. Utiyama, L. Liu, K. Chen, and E. Sumita, "Learning deep transformer models for machine translation," in Proc. 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 2019, pp. 1810 1822.
11. J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," Nature, vol. 596, no. 7873, pp. 583-589, Aug. 2021.
12. M. Fernandez-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" Journal of Machine Learning Research, vol. 15, no. 1, pp. 3133-3181, 2014.
13. R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," Information Fusion, vol. 81, pp. 84-90, May 2022.
14. S. O. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," in Proc. AAAI Conf. Artificial Intelligence, vol. 35, no. 8, pp. 6679-6687, 2021.
15. A. Guo and F. Berkhahn, "Entity embeddings of categorical variables," arXiv preprint arXiv:1604.06737, 2016.
16. S. Popov, S. Morozov, and A. Babenko, "Neural oblivious decision ensembles for deep learning on tabular data," in Proc. 8th Int. Conf. Learning Representations (ICLR), Addis Ababa, Ethiopia, 2020.
17. L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why tree-based models still outperform deep learning on tabular data," Advances in Neural Information Processing Systems, vol. 35, 2022.
18. X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "TabTransformer: Tabular data modelling using contextual embeddings," arXiv preprint arXiv:2012.06678, 2020.
19. N. Hollmann, S. Muller, K. Eggensperger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in Proc. 11th Int. Conf. Learning Representations (ICLR), Kigali, Rwanda, 2023.
20. J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, no. 1, pp. 281-305, 2012.
21. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Anchorage, AK, USA, 2019, pp. 2623-2631.
22. R. K. Pace and R. Barry, "Sparse spatial autoregressions," Statistics and Probability Letters, vol. 33, no. 3, pp. 291-297, May 1997.
23. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
24. J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics, Minneapolis, MN, USA, 2019, pp. 4171-4186.
25. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. 7th Int. Conf. Learning Representations (ICLR), New Orleans, LA, USA, 2019.